

Datsci 224
**Understanding Artificial Intelligence:
From Mathematics to Applications**

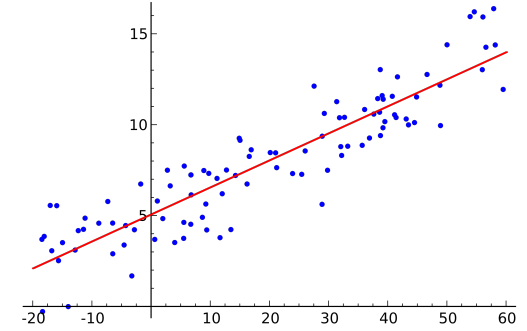
Lecture 2:
*Linear Regression and Uncertainty
Quantification*

Jean Feng
Department of Epidemiology and Biostatistics

Announcements

- Any questions from last lecture or lab?
- Homework #1 is due in one week.

Linear regression

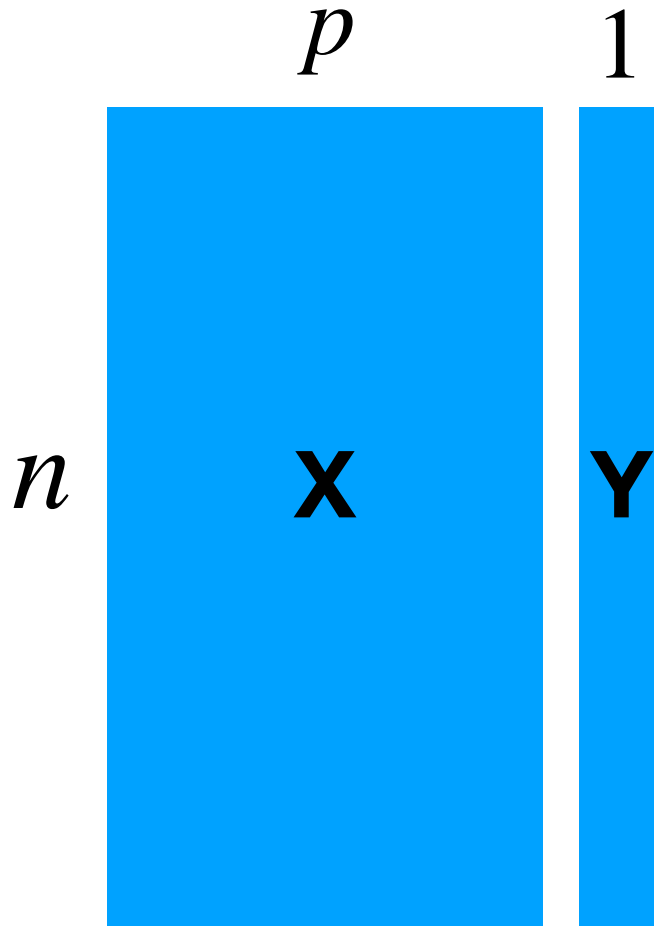


- Linear regression is the foundation of much of statistics and machine learning.
- In practice, linear regression (and generalized linear models) are some of the most commonly used algorithms.
- Linear regression is also a component in many machine learning algorithms, including:
 - Deep learning
 - Polynomial regression
- Many of the mathematical techniques that underly linear regression are fundamental to other machine learning algorithms.

Lecture Outline

- **Review of linear algebra**
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

Representing the data



Definition 2.1 (Matrix). With $m, n \in \mathbb{N}$ a real-valued (m, n) matrix A is an $m \cdot n$ -tuple of elements a_{ij} , $i = 1, \dots, m$, $j = 1, \dots, n$, which is ordered according to a rectangular scheme consisting of m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad a_{ij} \in \mathbb{R}. \quad (2.11)$$

Matrices

The sum of two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times n}$ is defined as the element-wise sum, i.e.,

$$\mathbf{A} + \mathbf{B} := \begin{bmatrix} a_{11} + b_{11} & \cdots & a_{1n} + b_{1n} \\ \vdots & & \vdots \\ a_{m1} + b_{m1} & \cdots & a_{mn} + b_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}. \quad (2.12)$$

For matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times k}$, the elements c_{ij} of the product $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times k}$ are computed as

$$c_{ij} = \sum_{l=1}^n a_{il} b_{lj}, \quad i = 1, \dots, m, \quad j = 1, \dots, k. \quad (2.13)$$

Dimensions must match to multiply two matrices:

$$\underbrace{\mathbf{A}}_{n \times k} \underbrace{\mathbf{B}}_{k \times m} = \underbrace{\mathbf{C}}_{n \times m}$$

This is an extremely useful trick to check your math!

The Most Special Matrix

Definition 2.2 (Identity Matrix). In $\mathbb{R}^{n \times n}$, we define the *identity matrix*

$$\mathbf{I}_n := \begin{bmatrix} 1 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \in \mathbb{R}^{n \times n} \quad (2.17)$$

Multiplication with an identity matrix:

For any matrix $A \in \mathbb{R}^{m \times n}$, we have $AI_n = I_m A = A$.

Matrix transformations

Definition 2.3 (Inverse). Consider a square matrix $A \in \mathbb{R}^{n \times n}$. Let matrix $B \in \mathbb{R}^{n \times n}$ have the property that $AB = I_n = BA$. B is called the *inverse* of A and denoted by A^{-1} .

Example 2.4 (Inverse Matrix)

The matrices

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 4 & 4 & 5 \\ 6 & 7 & 7 \end{bmatrix}, \quad B = \begin{bmatrix} -7 & -7 & 6 \\ 2 & 1 & -1 \\ 4 & 5 & -4 \end{bmatrix} \quad (2.25)$$

are inverse to each other since $AB = I = BA$.

Not all matrices have inverses!!

Matrix A is invertible $\iff$ Determinant of A is nonzero. $\iff$ Eigenvalues of A are nonzero.

Matrix transformations

Definition 2.4 (Transpose). For $A \in \mathbb{R}^{m \times n}$ the matrix $B \in \mathbb{R}^{n \times m}$ with $b_{ij} = a_{ji}$ is called the *transpose* of A . We write $B = A^\top$.

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \quad A^\top = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}$$

Matrices appear everywhere in Linear Regression

- **Prediction for a single observation from a linear model:**

- For observation $x \in \mathbb{R}^{p \times 1}$, coefficients $\beta \in \mathbb{R}^{p \times 1}$, and intercept β_0 :

$$f(x) = \sum_{j=1}^p x_j \beta_j + \beta_0 = x^\top \beta + \beta_0$$

- **Predictions for n observations from a linear model**

$$\bullet \quad f(x^{(i)}) = \sum_{j=1}^p x_j^{(i)} \beta_j + \beta_0 \quad \forall i = 1, \dots, n \quad \Longleftrightarrow \quad \begin{pmatrix} f(x^{(1)}) \\ \vdots \\ f(x^{(n)}) \end{pmatrix} = X\beta + \beta_0$$

- If we define the augmented matrix $\tilde{X} = \begin{pmatrix} X & \vec{1} \end{pmatrix}$ and $\tilde{\beta} = \begin{pmatrix} \beta & \beta_0 \end{pmatrix}$, then it is equivalent to

write $\begin{pmatrix} f(x^{(1)}) \\ \vdots \\ f(x^{(n)}) \end{pmatrix} = \tilde{X}\tilde{\beta}.$

Vector norms

- **Norms** quantify the size of a vector.

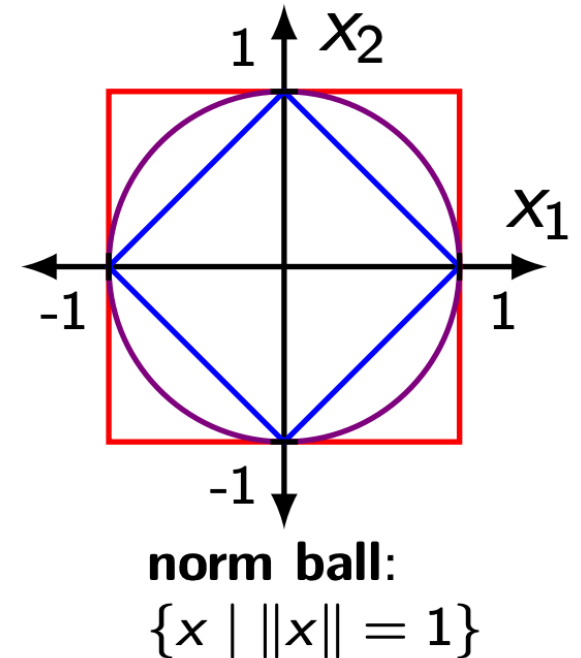
$$\|x\|_1 = |x_1| + \cdots + |x_n| \quad (\text{the 1-norm})$$

$$\|x\|_2 = \sqrt{x_1^2 + \cdots + x_n^2} \quad (\text{the 2-norm})$$

$$\|x\|_\infty = \max_{1 \leq i \leq n} |x_i| \quad (\text{max-norm})$$

How else can you express the L2 norm?

$$\|x\|_2^2 = x^\top x$$



Where do you remember seeing norms in ML?

- The mean squared error loss can be expressed using the L2 norm: $\|Y - (X\beta + \beta_0)\|^2$
- Ridge penalty: $\|\beta\|_2^2$
- Lasso penalty: $\|\beta\|_1$
- ...

Matrix properties

- Properties:

- $A + B = B + A$

Commutative property

- $A(BC) = (AB)C$

Associative property

- $A(B + C) = AB + AC$

Distributive property

- $A = (A^T)^T$

- $(AB)^T = B^T A^T$

- ...

- Commutativity does not necessarily hold for matrix products! It is often the case that $AB \neq BA$.



Getting overwhelmed?

The Matrix Cookbook

[<http://matrixcookbook.com>]

Kaare Brandt Petersen
Michael Syskind Pedersen

VERSION: NOVEMBER 15, 2012

<https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>

Matrix calculus

We will be using the
denominator-layout notation

$$\frac{\partial y}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \\ \vdots \\ \frac{\partial y}{\partial x_n} \end{bmatrix}.$$

$$\frac{\partial \mathbf{y}}{\partial x} = \begin{bmatrix} \frac{\partial y_1}{\partial x} & \frac{\partial y_2}{\partial x} & \cdots & \frac{\partial y_m}{\partial x} \end{bmatrix}.$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \frac{\partial y_2}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}.$$

$$\frac{\partial y}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial y}{\partial x_{11}} & \frac{\partial y}{\partial x_{12}} & \cdots & \frac{\partial y}{\partial x_{1q}} \\ \frac{\partial y}{\partial x_{21}} & \frac{\partial y}{\partial x_{22}} & \cdots & \frac{\partial y}{\partial x_{2q}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y}{\partial x_{p1}} & \frac{\partial y}{\partial x_{p2}} & \cdots & \frac{\partial y}{\partial x_{pq}} \end{bmatrix}.$$

Handy differentiation rules

$$1. \frac{\partial \mathbf{a}^T \mathbf{z}}{\partial \mathbf{z}} = \mathbf{a}$$

$$2. \frac{\partial M \mathbf{z}}{\partial \mathbf{z}} = M^T$$

$$3. \frac{\partial \mathbf{z}^T M \mathbf{z}}{\partial \mathbf{z}} = (M + M^T) \mathbf{z}$$

Chain Rule: For function $y = g(h(k(x)))$,
we have $\frac{\partial y}{\partial x} = \frac{\partial k}{\partial x} \frac{\partial h}{\partial k} \frac{\partial g}{\partial h}$.

*You can prove
these to yourself!*

Quiz

- **Q:** What is the $\frac{\partial}{\partial \mathbf{z}} \|\mathbf{z}\|_2^2$?
- **A:** $\mathbf{z}$
- **B:** $2\mathbf{z}$
- **C:** $\|\mathbf{z}\|_1$
- **D:** $\|\mathbf{z}\|_2^2$

Handy differentiation rules

1. $\frac{\partial \mathbf{a}^T \mathbf{z}}{\partial \mathbf{z}} = \mathbf{a}$
2. $\frac{\partial M\mathbf{z}}{\partial \mathbf{z}} = M^T$
3. $\frac{\partial \mathbf{z}^T M \mathbf{z}}{\partial \mathbf{z}} = (M + M^T)\mathbf{z}$

Lecture Outline

- Review of linear algebra
 - **Vectorizing code**
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

Code efficiency

- Example task: Calculate the mean squared error for a ML algorithm on a test dataset.

```
n = 10000  
y_hat = np.random.normal(size=n)  
y = np.random.normal(size=n)
```

```
mse = np.mean(np.power(y - y_hat, 2))
```

Time: 0.0001 seconds

versus

```
tot_mse = 0  
for i in range(n):  
    tot_mse += np.power(y[i] - y_hat[i], 2)  
mse = tot_mse/n
```

Time: 0.0137 seconds

Leveraging linear algebra to speed up code

- **Vectorized code** refers to operations that are performed on multiple components of a vector at the same time (in one statement).
- Modern CPUs and GPUs contain vector processing units. Vector processors are specialized processors designed to perform the same operation on multiple data elements simultaneously.
 - If you want to take advantage of vector processors, you have to vectorize your code. ***Avoid loops whenever possible.***
- To vectorize your code, the data needs to be organized in vectors/matrices/arrays. ***The more linear algebra you know, the better you'll be at vectorizing your code!***

Linear regression example

- Getting predictions from linear regression:

```
y_hat = np.zeros(n)
for i in range(n):
    for j in range(p):
        y_hat[i] += X[i,j] * beta[j]
```

- How can we make it more efficient?

```
y_hat = []
for i in range(n):
    y_hat.append(np.dot(x[i], beta))
```

```
y_hat = np.matmul(X, beta)
```

*Vectorized code is
also much shorter.*

*Fewer chances for
bugs.*

(Also, what's wrong with `y_hat = x * beta` ?)

Lecture Outline

- Review of linear algebra
 - Vectorizing code
- **Assumptions**
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

What does Linear Regression assume?

- There are many ways to formulate/motivate linear regression.

- Approach 1:

- Suppose $Y = X^\top \beta + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$.

Assume a probability model

- Approach 2:

- Suppose $\mathbb{E}[Y | X = x] = x^\top \beta$ and $\text{Var}(\epsilon | X) = \sigma^2$.

Assume a model for the mean and variance

- Approach 3:

- Find β that minimizes $\|Y - X\beta\|_2^2$.

Find the best linear approximation

How do assumptions between the approaches compare?

Why is Ordinary Least Squares (OLS) the solution to Approach 1?

- Approach 1: Suppose $Y = X^\top \beta + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$.
- To solve for β , we use Maximum Likelihood Estimation (MLE):

$$\arg \max_{\beta} \sum_{i=1}^n \log p(y_i, x_i; \beta, \sigma^2)$$

$$\iff \arg \max_{\beta} \sum_{i=1}^n \log p(y_i | x_i; \beta, \sigma^2) + \log p(x_i)$$

$$\iff \arg \max_{\beta} \sum_{i=1}^n \log p(y_i | x_i; \beta, \sigma^2)$$

$$\iff \arg \max_{\beta} \sum_{i=1}^n \log p(\epsilon_i; \beta, \sigma^2)$$

Why is Ordinary Least Squares (OLS) the solution to Approach 1?

Massaging further... $\arg \max_{\beta} \sum_{i=1}^n \log p(\epsilon_i; \beta, \sigma^2)$

$$\Leftrightarrow \arg \max_{\beta} \sum_{i=1}^n \log \left[\frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{(y_i - x_i^{\top} \beta)^2}{2\sigma^2} \right) \right]$$

Recall that for normal distribution $\epsilon \sim N(0, \sigma^2)$, we have $p(\epsilon; \sigma) = \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{\epsilon^2}{2\sigma^2} \right)$.

$$\Leftrightarrow \arg \max_{\beta} \sum_{i=1}^n \left[\log \frac{1}{\sigma \sqrt{2\pi}} - \frac{(y_i - x_i^{\top} \beta)^2}{2\sigma^2} \right]$$

$$\Leftrightarrow \arg \max_{\beta} - \sum_{i=1}^n \frac{(y_i - x_i^{\top} \beta)^2}{2\sigma^2}$$

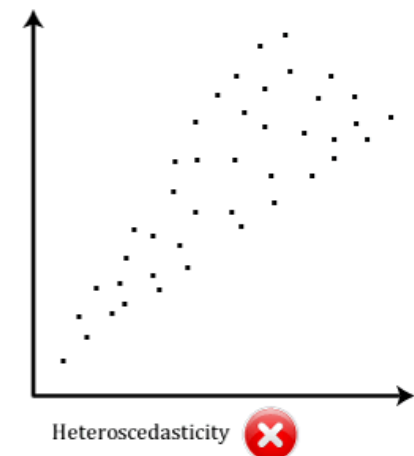
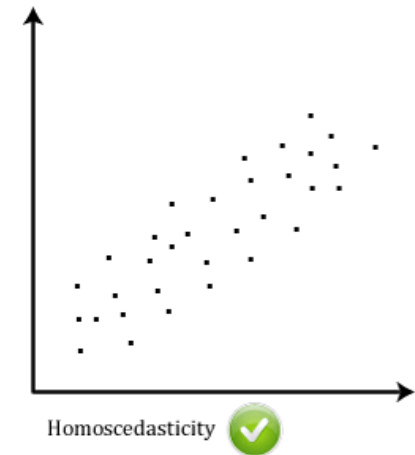
$$\Leftrightarrow \arg \min_{\beta} \sum_{i=1}^n \frac{(y_i - x_i^{\top} \beta)^2}{2\sigma^2}$$

$$\Leftrightarrow \arg \min_{\beta} \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2$$

Just minimize the mean squared error!

Other implicit assumptions of OLS

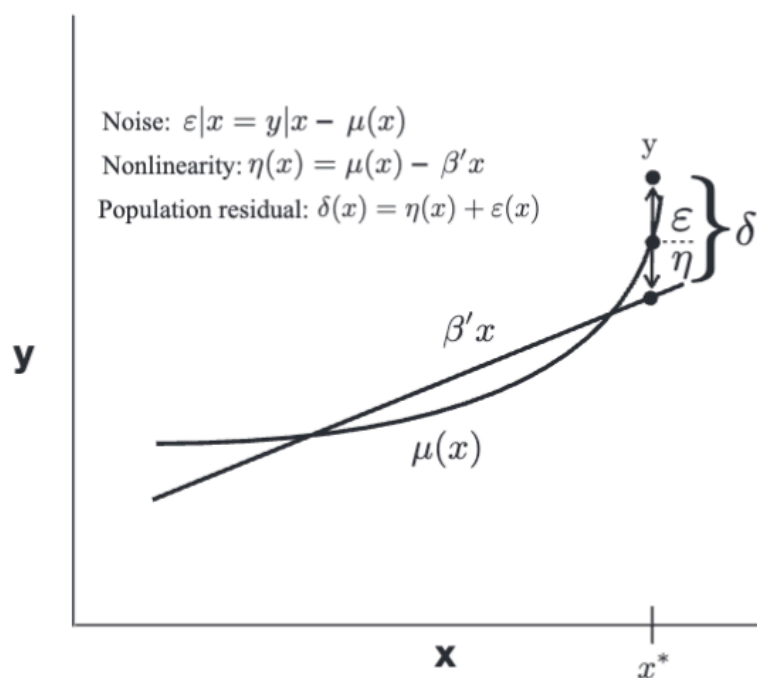
- **Homoskedasticity:** The error term has the same variance σ^2 in each observation
 - Opposite: **heteroskedasticity**
- **Independent errors:** Errors are independent, i.e. $\epsilon_i \perp \epsilon_j \forall i \neq j$.
 - Opposite: **Correlated errors**



🤔 *What extensions of OLS can be applied if errors are heteroskedastic or correlated?*

Assumption lean regression

- By viewing OLS as a method to estimate the best linear approximation, we make very few assumptions.
- However, one needs to be aware of when the best linear approximation is misleading.



Note of caution...

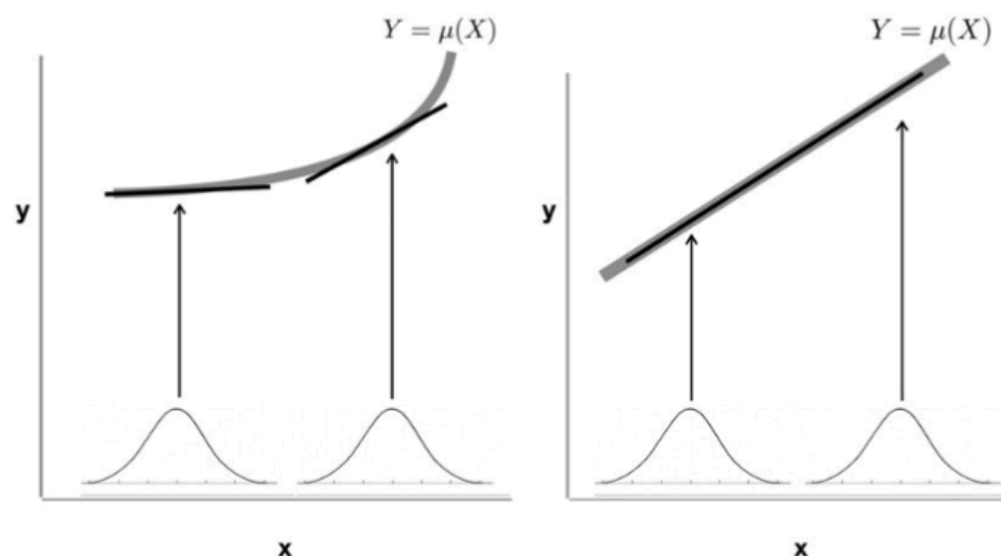


Figure 1. A population decomposition of $Y|X$ using the best linear approximation.

Assumption Lean Regression

Richard Berk^{a,b}, Andreas Buja^b, Lawrence Brown^b, Edward George^b, Arun Kumar Kuchibhotla^b, Weijie Su^b, and Linda Zhao^b

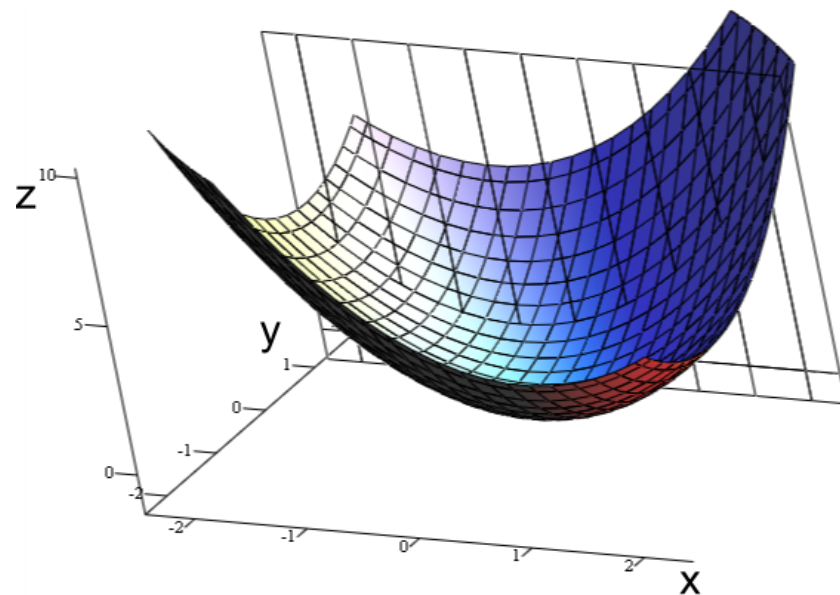
Lecture Outline

- Review of linear algebra
 - Vectorizing code
- Assumptions
- **Solving Linear Regression**
 - **Closed-form solution**
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

Solving Linear Regression

$$\min_{\beta} \| Y - X\beta \|_2^2$$

This is a convex
unconstrained optimization
problem (if there are enough
observations and the
variables are not collinear).



Linear regression: closed-form solution

- **Recall:** For an unconstrained convex optimization problem, θ is the solution to $\min_{\theta \in \mathbb{R}^p} f(\theta)$ if and only if it satisfies $\nabla_{\theta} f(\theta) = 0$.

- **Step 1:** Deriving the gradient

$$\nabla_{\beta} \|Y - X\beta\|^2$$

$$= \nabla_{\beta} (Y - X\beta)^{\top} (Y - X\beta)$$

$$= \left[\nabla_{\beta} X\beta \right] \left[\nabla_{X\beta} (Y - X\beta)^{\top} (Y - X\beta) \right]$$

$$= -2X^{\top} (Y - X\beta)$$

Handy differentiation rules

$$1. \frac{\partial \mathbf{a}^{\top} \mathbf{z}}{\partial \mathbf{z}} = \mathbf{a}$$

$$2. \frac{\partial M\mathbf{z}}{\partial \mathbf{z}} = M^{\top}$$

$$3. \frac{\partial \mathbf{z}^{\top} M\mathbf{z}}{\partial \mathbf{z}} = (M + M^{\top})\mathbf{z}$$

Chain rule

Linear regression: closed-form solution

- **Step 2:** Set the gradient to zero and solve for β :

$$-2X^T(Y - X\hat{\beta}) = 0$$

$$X^TY - X^TX\hat{\beta} = 0$$

$$X^TX\hat{\beta} = X^TY$$

$$\hat{\beta} = (X^TX)^{-1}X^TY$$

*Note: The inverse matrix does not always exist!
It generally does when (i) there are enough
observations and (ii) variables are not collinear.*

Lecture Outline

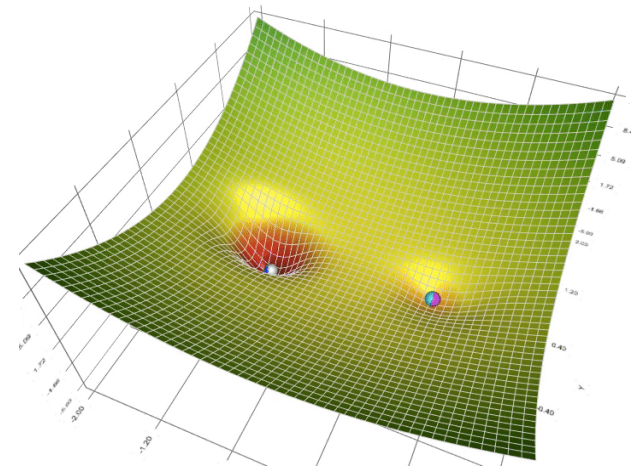
- Review of linear algebra
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - **Gradient Descent**
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

When there is no closed-form solution...

- Most of the time, there is no closed-form solution for an optimization problem.
- Examples:
 - Logistic regression (your homework!)
 - Neural networks
 - Gradient boosted trees
- In this case, we try to find the solution using an *iterative* optimization procedure.

Gradient descent

- Gradient descent is a **first-order** optimization procedure, i.e. the optimization procedure only requires the first-derivative/gradient.
- Gradient descent (GD) and its variants are used nearly everywhere in machine learning.
- Variants of GD:
 - Stochastic gradient descent
 - Minibatch gradient descent
 - Adam



Gradient descent algorithm

Suppose we want to solve $\min_{\theta} f(\theta)$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1, 2, \dots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

$$\text{If } \frac{f(\theta_{(t-1)}) - f(\theta_{(t)})}{|f(\theta_{(t-1)})|} < \epsilon:$$

Return $\theta_{(t)}$

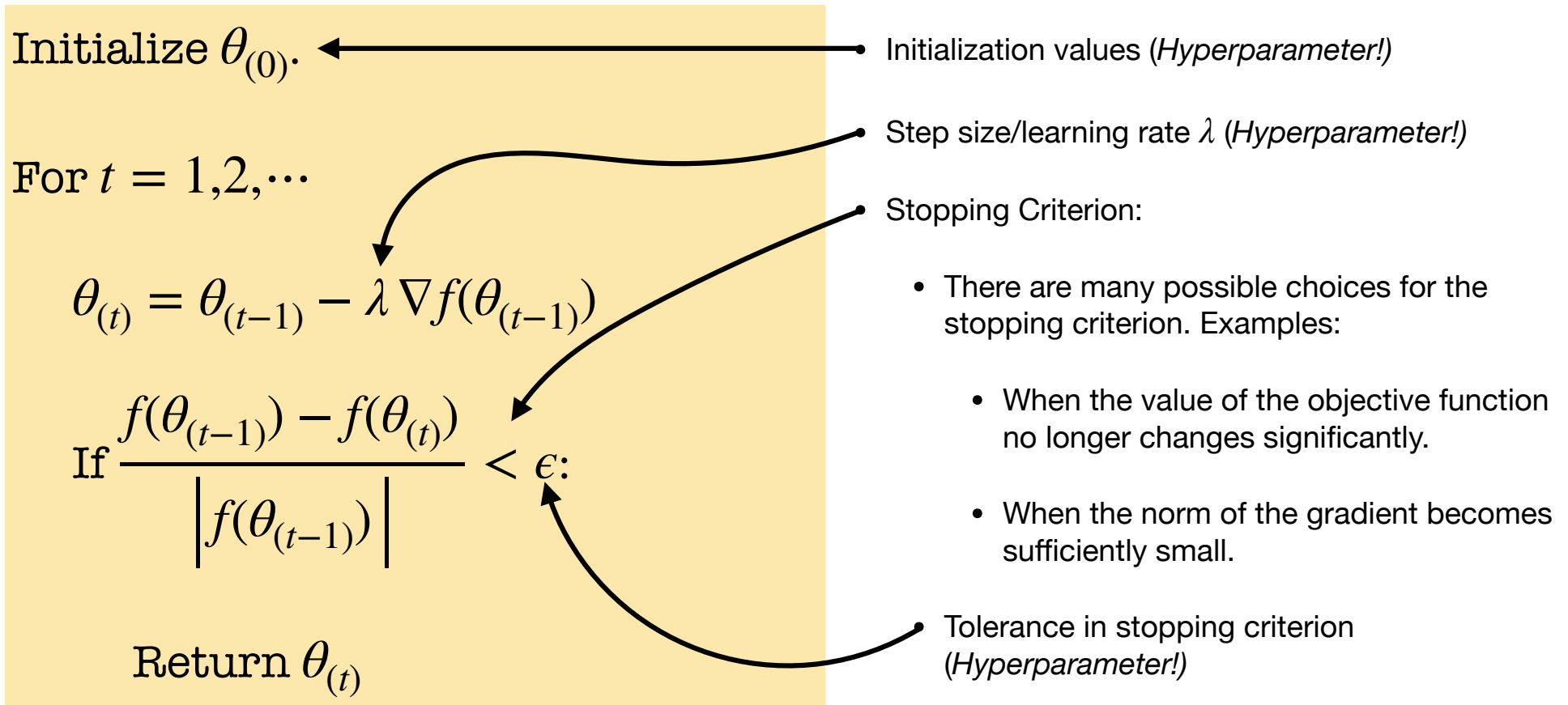
Initialization

Gradient update — shift θ a little bit in the direction of the **negative** gradient

Why?

Stopping criterion

Gradient descent algorithm components



Good news! When the problem is convex, it generally doesn't matter where you initialize. There is only one solution and you are guaranteed to converge to it if you run GD for sufficiently long with a small enough step size.

Gradient descent algorithm components

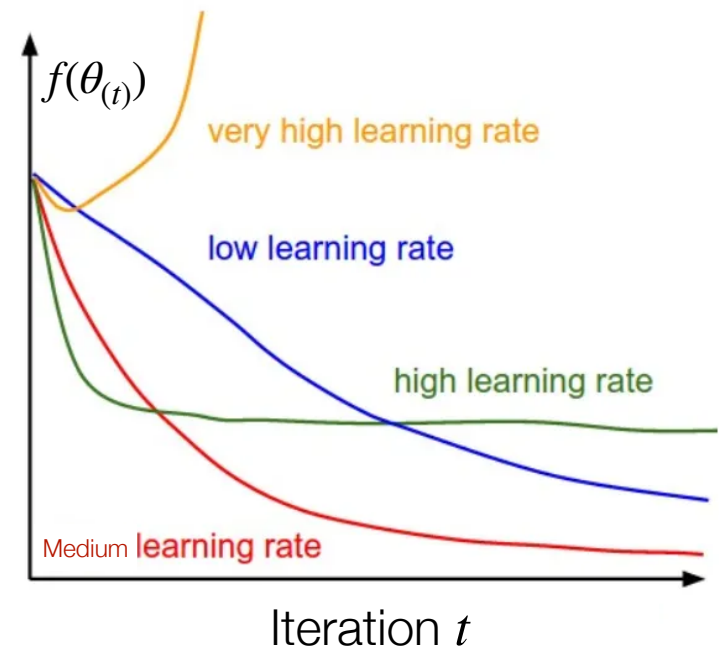
Initialize $\theta_{(0)}$.

For $t = 1, 2, \dots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

If $\frac{f(\theta_{(t-1)}) - f(\theta_{(t)})}{|f(\theta_{(t-1)})|} < \epsilon$:

Return $\theta_{(t)}$



Gradient descent algorithm: Let's apply it to linear regression

Initialize $\theta_{(0)}$.

For $t = 1, 2, \dots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

$$\text{If } \frac{f(\theta_{(t)}) - f(\theta_{(t-1)})}{|f(\theta_{(t-1)})|} < \epsilon:$$

Return $\theta_{(t)}$

- What is f and θ ?

- $f(\beta) = \|Y - X\beta\|^2$

- β = coefficients and intercept of linear regression model.

- What is the gradient?

$$\nabla_{\beta} \|Y - X\beta\|^2 = -2X^{\top} (Y - X\beta)$$

Gradient descent algorithm: Let's apply it to linear regression

Initialize $\theta_{(0)}$.

For $t = 1, 2, \dots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

$$\text{If } \frac{f(\theta_{(t)}) - f(\theta_{(t-1)})}{|f(\theta_{(t-1)})|} < \epsilon:$$

Return $\theta_{(t)}$

Initialize $\beta_{(0)}$.

For $t = 1, 2, \dots$

$$\beta_{(t)} = \beta_{(t-1)} + 2\lambda X^T (Y - X\beta)$$

$$\text{If } \frac{\|Y - X\beta_{(t-1)}\|^2 - \|Y - X\beta_{(t)}\|^2}{\|Y - X\beta_{(t-1)}\|^2} < \epsilon:$$

Return $\beta_{(t)}$

Lecture Outline

- Review of linear algebra
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- **Estimand vs estimate**
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- Nonlinear Regression

Estimands vs Estimates

- Let our target population be denoted P . Given n samples from the population P , let us denote the **empirical distribution** of these sample as $\hat{P}_n$.
- An **estimand** is defined as a function of the population, i.e. $\Psi(P)$.
- An **estimate** is our guess of the estimand's value given $\hat{P}_n$, i.e. $\hat{\Psi}(\hat{P}_n)$. We often call $\hat{\Psi}$ an **estimation procedure** or an **estimator**.
- **Example 1:**
 - **Estimand** is the population mean $\mu(P) = \mathbb{E}_P[X]$.
 - **Estimate** is the empirical mean $\hat{\mu}(\hat{P}_n) = \frac{1}{n} \sum_{i=1}^n X_i$.

Usually, our estimate is not exactly equal to our estimand!

Estimands vs Estimates

- **Example 2:**

- **Estimand** is the best linear model for the population
 $\theta(P) = \mathbb{E}_P \left[(X^\top X)^{-1} X^\top Y \right].$
- **Estimate** is the linear model that we fit on the data $\hat{\theta}(\hat{P}_n) = (X^\top X)^{-1} X^\top Y.$

- **Example 3:**

- **Estimand** is the expected outcome for a patient with variables x , i.e.
 $f(x; P) = \mathbb{E}_P \left[Y | X = x \right].$
- **Estimate** is the prediction from a machine learning algorithm $\hat{f}(x; \hat{P}_n).$

How good is an estimate?

- Different metrics for characterizing the performance of an estimation procedure:
 - **Bias:** The average difference between my estimate and the estimand, defined by $\mathbb{E} \left[\hat{\Psi}(\hat{P}_n) - \Psi(P) \right]$.
 - **Asymptotic Bias:** As n increases, what is the average difference between my estimate and the estimand?
 - **Variance:** The variance of my estimate $Var \left(\hat{\Psi}(\hat{P}_n) \right)$ (i.e. how variable is my estimate?)
 - **Asymptotic Variance:** As n increases, what is the variance of my estimate?
- Usually there is a **tradeoff**. Something can have large bias but small variance or small variance but large bias.

Example: population mean

- **Estimand** is the population mean $\mu(P) = \mathbb{E}_P[X]$.

- **Estimator options:**

- **A:** The constant 14

What is the bias and variance of these estimators?

*This estimate is **biased** but **zero variance**.*

- **B:** The empirical mean

$$\hat{\mu}(\hat{P}_n) = \frac{1}{n} \sum_{i=1}^n X_i.$$

*This estimate is **unbiased** for any n , but has **non-zero variance**.*

Example: linear regression

- **Estimand** is the best linear model for the population
$$\theta(P) = \mathbb{E}_P \left[(X^\top X)^{-1} X^\top Y \right].$$

- **Estimator options:**

What is the bias and variance of these estimators?

- **A:** Ordinary least squares (OLS)

$$\hat{\theta}(\hat{P}_n) = (X^\top X)^{-1} X^\top Y.$$

*This estimate is **unbiased** (for sufficiently large n). Moreover, according to the Gauss-Markov Theorem, OLS has the **smallest variance** among all linear unbiased estimates.*

- **B:** Ridge regression

$$\hat{\theta}_{ridge}(\hat{P}_n) = (X^\top X + \lambda I)^{-1} X^\top Y$$

*This estimate is **not unbiased**, but it does have **lower variance** than OLS.*

Mean Squared Error

- Another way to measure performance of an estimator is its **Mean Squared Error (MSE)**, the average squared difference between my estimate and the estimand.

The MSE is defined as $\mathbb{E} \left[\left(\hat{\Psi}(\hat{P}_n) - \Psi(P) \right)^2 \right]$.

- How is MSE related to Bias and Variance?

$$\begin{aligned} \mathbb{E} \left[\left(\hat{\Psi}(\hat{P}_n) - \Psi(P) \right)^2 \right] &= \mathbb{E} \left[\hat{\Psi}(\hat{P}_n)^2 - 2\hat{\Psi}(\hat{P}_n)\Psi(P) + \Psi(P)^2 \right] \\ &= \underbrace{\mathbb{E} \left[\hat{\Psi}(\hat{P}_n)^2 \right] - \mathbb{E} \left[\hat{\Psi}(\hat{P}_n) \right]^2}_{\text{Var}(\hat{\Psi})} + \underbrace{\mathbb{E} \left[\hat{\Psi}(\hat{P}_n) \right]^2 - 2\mathbb{E} \left[\hat{\Psi}(\hat{P}_n) \right] \Psi(P) + \Psi(P)^2}_{\text{Bias}^2 = \left(\mathbb{E} \left[\hat{\Psi}(\hat{P}_n) - \Psi(P) \right] \right)^2} \end{aligned}$$

So if we want small MSE, we want small bias *and* small variance.

Example: population mean

- **Estimand** is the population mean $\mu(P) = \mathbb{E}_P[X]$.

- **Estimator options:**

How do the MSEs compare?

- **A:** The constant 14

Large MSE

- **B:** The empirical mean

$$\hat{\mu}(\hat{P}_n) = \frac{1}{n} \sum_{i=1}^n X_i.$$

Smaller MSE, will converge to zero with sufficiently large n

Example: linear regression

- **Estimand** is the best linear model for the population
$$\theta(P) = \mathbb{E}_P \left[(X^\top X)^{-1} X^\top Y \right].$$

- **Estimator options:**

How do the MSEs compare?

- **A:** Ordinary least squares (OLS)

$$\hat{\theta}(\hat{P}_n) = (X^\top X)^{-1} X^\top Y.$$

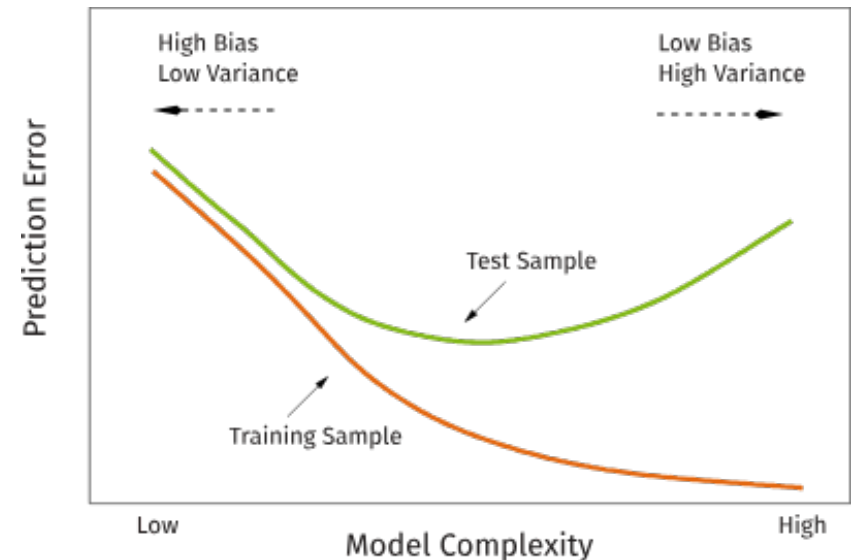
Ridge can achieve smaller MSE than OLS for properly chosen λ , particularly when n is small.

- **B:** Ridge regression

$$\hat{\theta}_{ridge}(\hat{P}_n) = (X^\top X + \lambda I)^{-1} X^\top Y$$

Evaluating an estimator

- Do we care more about bias, variance, or mean squared error? 🤔
- Depends on the problem setting!
- Bias is often viewed as very scary and sometimes viewed as our number one priority. That shouldn't always be the case!

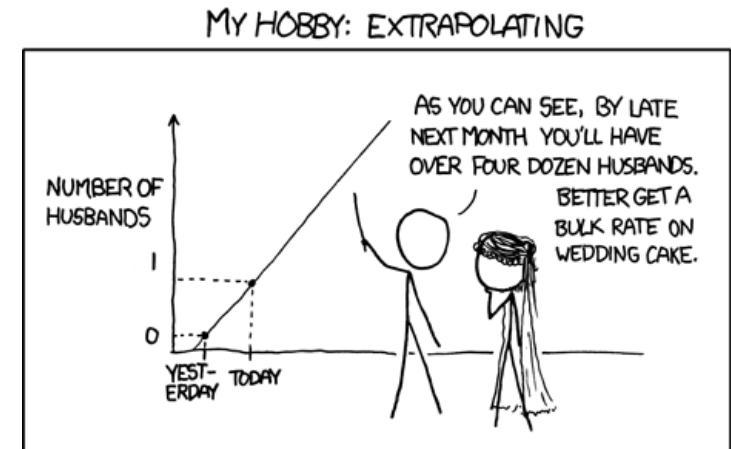
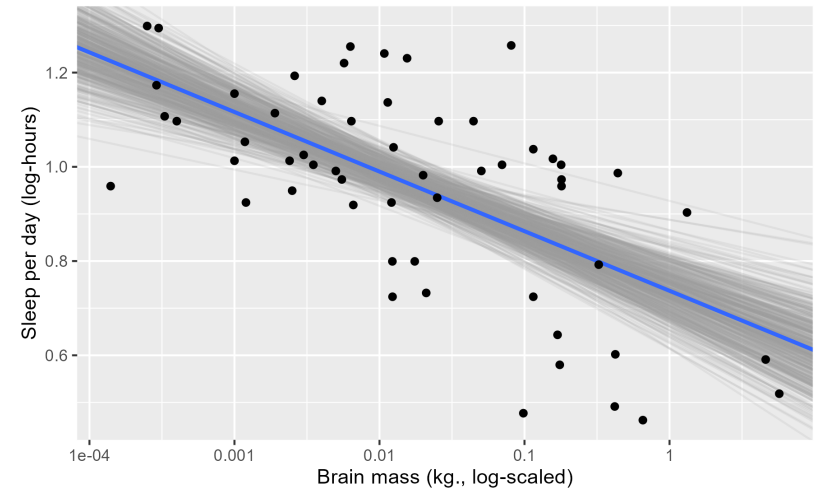


Lecture Outline

- Review of linear algebra
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- **Uncertainty Quantification**
 - **Prediction interval vs confidence interval**
 - Confidence intervals
- Nonlinear Regression

Uncertainty quantification

- **Different types of uncertainty:**
 - There is inherent random variation, e.g.
 - The result of a coin flip
 - Whether a patient will get cancer
 - There is uncertainty in our estimate.
 - There is uncertainty about the right model to use.
 - There is uncertainty when predicting for observations that are far from the training data
- *When talking about uncertainty quantification in ML, be specific!*



Two examples of uncertainty quantification

A **prediction interval** is a range of values that is likely to contain the value of a ***new observation***.

For a population where we just observe X , a prediction interval is

$$\Pr(X \in PI) \geq 1 - \alpha.$$

For regression problem with (X, Y) , a prediction interval is

$$\Pr(Y \in PI(x) \mid X = x) \geq 1 - \alpha.$$

A **confidence interval** is a range outputted by a procedure that is likely to contain an ***estimand***.

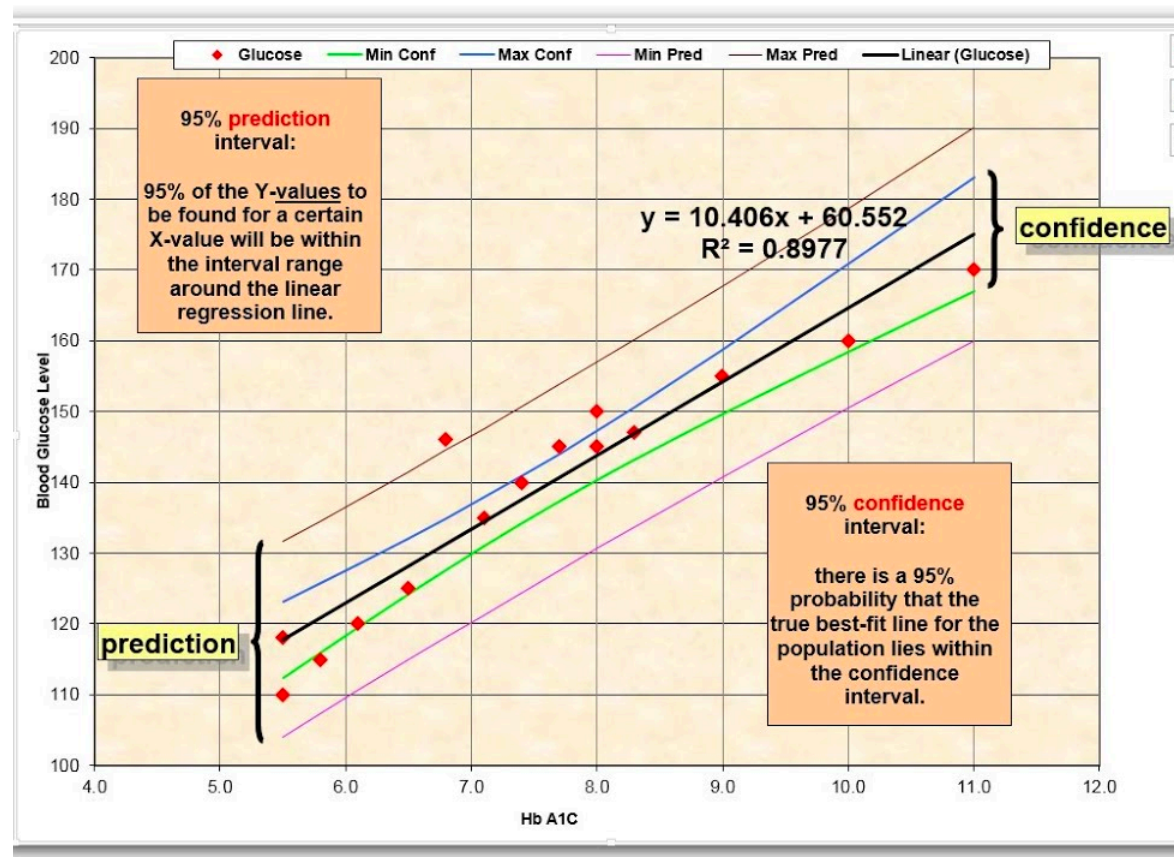
A CI for the population mean:
$$\Pr(\mu \in CI(\hat{P}_n)) \geq 1 - \alpha$$

CI for the conditional mean:
$$\Pr(\mathbb{E}[Y \mid X = x] \in CI(x; \hat{P}_n)) \geq 1 - \alpha$$

Prediction intervals vs confidence intervals

Confidence intervals aim to describe properties of the true population.

Prediction intervals aim to describe what is likely to happen next.



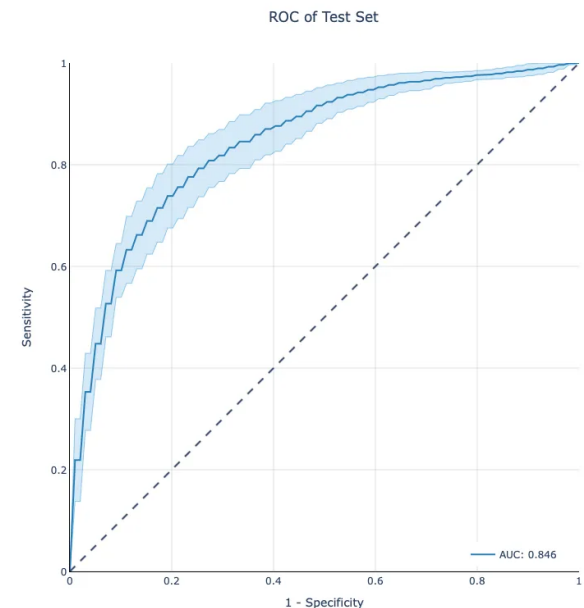
Because the value of a future observation is inherently noisy, prediction intervals are wider than confidence intervals.

Lecture Outline

- Review of linear algebra
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - **Confidence intervals**
- Nonlinear Regression

What quantities in ML do we usually want CIs for?

- Example estimands we are usually interested in:
 - Descriptive statistics, e.g. average age in a population
 - 0-1 accuracy of a classifier
 - True positive rate of a classifier
 - Area under the ROC curve (AUC) of a classifier
 - Mean squared error of a linear regression model
 - Parameters of a linear model



A statistical approach to model evaluations

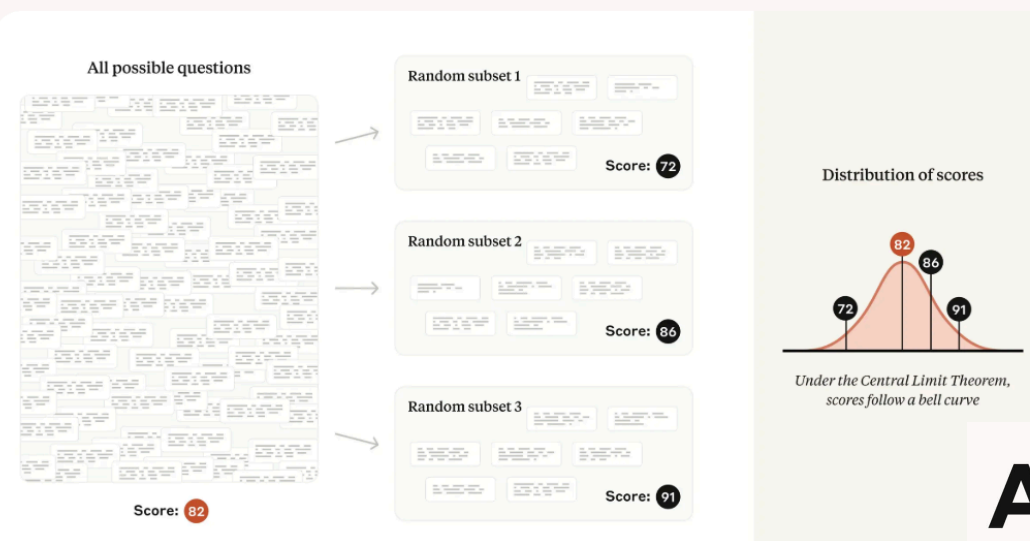
Nov 19, 2024

Read the paper

Suppose an AI model outperforms another model on a benchmark of interest—testing its general knowledge, for example, or its ability to solve computer-coding questions. Is the difference in capabilities real, or could one model simply have gotten lucky in the choice of questions on the benchmark?

With the amount of public interest in AI model evaluations—informally called “evals”—this question remains surprisingly understudied among the AI research community. This month, we published a [new research paper](#) that attempts to answer the question rigorously. Drawing on statistical theory and the experiment design literature, the paper makes a number of recommendations to the AI research community for reporting eval results in a scientifically informative way. In this post, we briefly go over the reporting recommendations, and the logic behind them.

Recommendation #1: Use the Central Limit Theorem



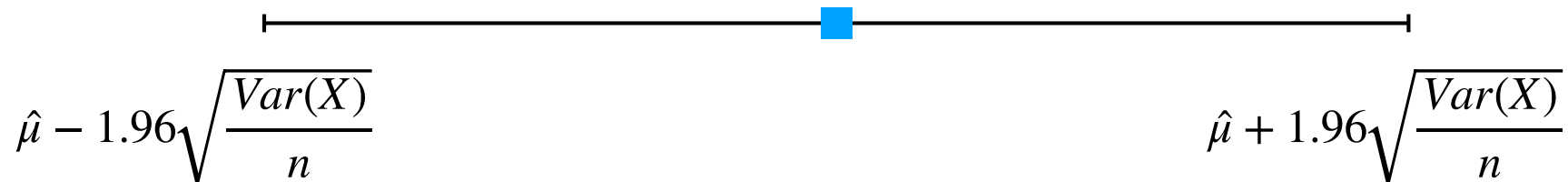
Confidence intervals

- A $(1 - \alpha)$ **confidence interval** $C(\hat{P}_n)$ satisfies the property that $\Pr(\Psi(P) \in C(\hat{P}_n)) \geq 1 - \alpha$.
- The **standard error** of an estimator is its standard deviation.
- We can analyze the theoretical properties of our estimator and deduce its behavior, often by analyzing its *asymptotic properties*.

Confidence intervals

- For many traditional statistical models, we know how to calculate confidence intervals.
- **Example:** Let the population mean be the estimand of interest.
 - Suppose we have an estimate of the population mean given n observations.
 - Under certain assumptions, we know that the 95% confidence interval for the population mean is

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i = \bar{X}$$



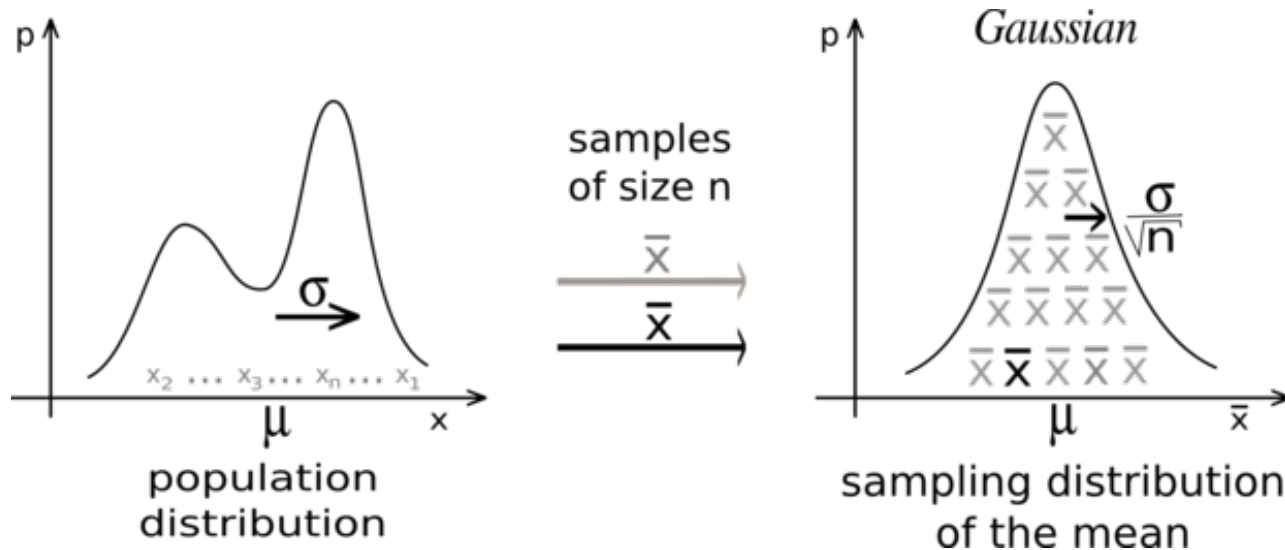
A horizontal line with a blue square in the center represents the confidence interval. The left endpoint is labeled $\hat{\mu} - 1.96\sqrt{\frac{\text{Var}(X)}{n}}$ and the right endpoint is labeled $\hat{\mu} + 1.96\sqrt{\frac{\text{Var}(X)}{n}}$. The blue square is positioned at the center of the line, corresponding to the sample mean $\hat{\mu}$.

Where did this formula come from?

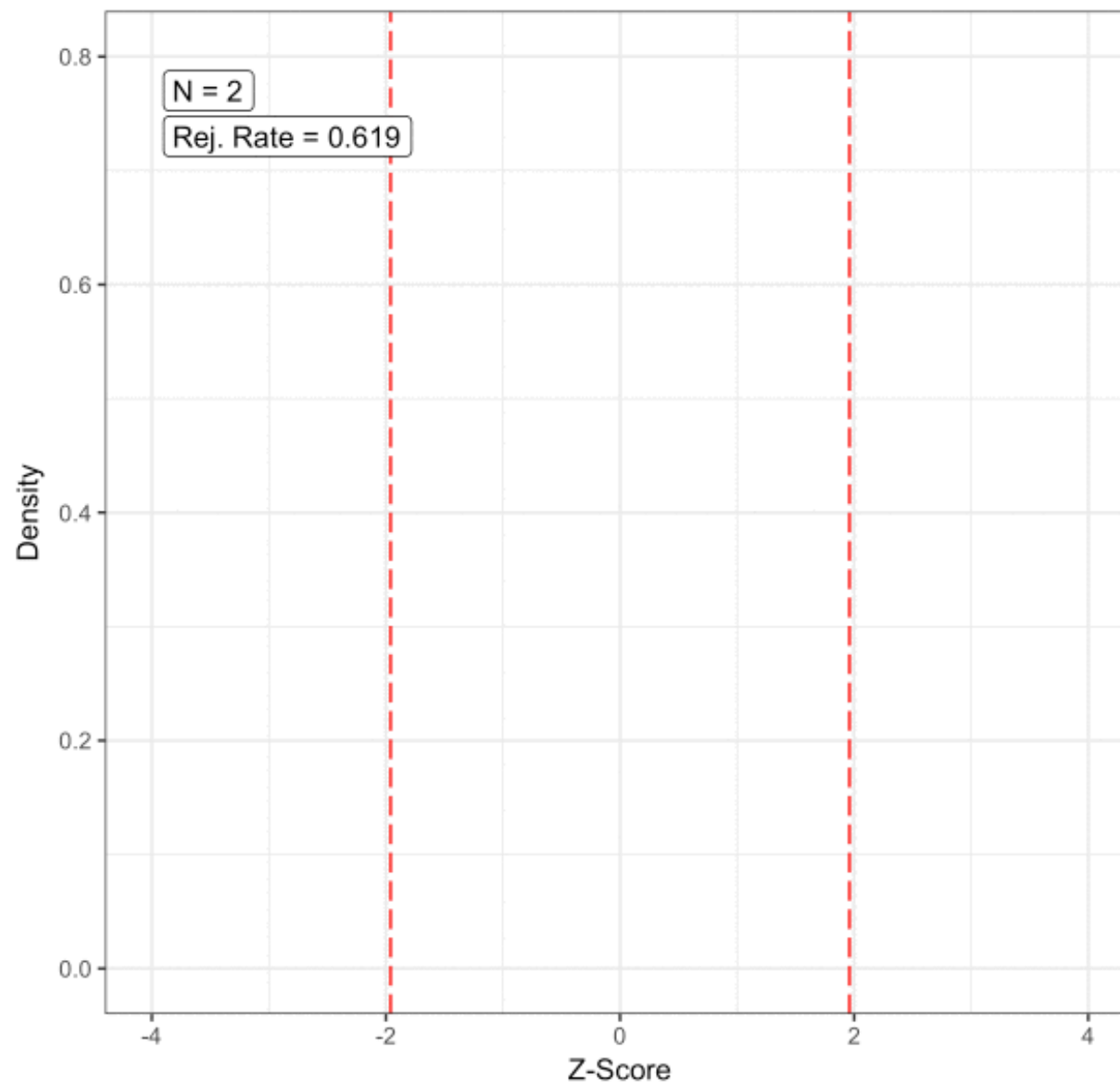
The central limit theorem

Suppose $X_1, X_2, \dots, X_n$ is a sequence of i.i.d. random variables with mean $\mathbb{E}[X] = \mu$ and $\text{Var}(X) = \sigma^2 < \infty$. As n approaches infinity, we have

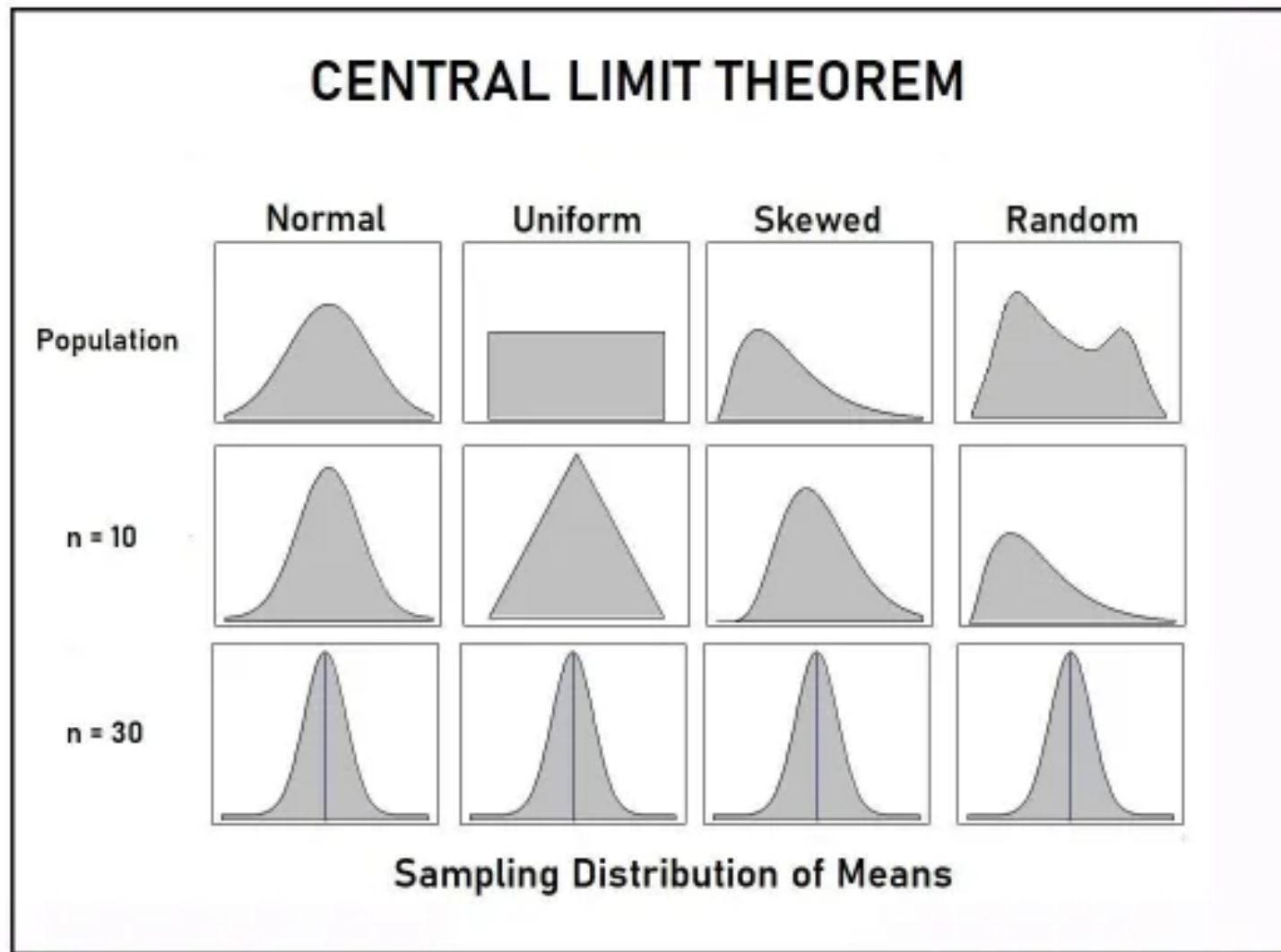
$$\sqrt{n} \frac{(\bar{X}_n - \mu)}{\sigma} \rightarrow_d N(0,1)$$



Histogram of Z-Scores from 10000 Distributions



The central limit theorem

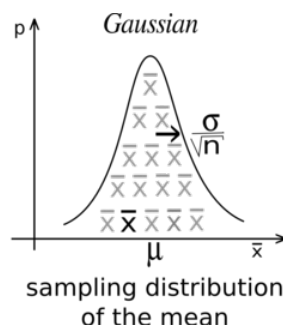


From CLT to Confidence intervals

The big idea: If we know that the difference between our estimator and estimand is (approximately) normally distributed, then we can construct an interval around our estimate such that the estimand will likely be in!

$$\underbrace{\sqrt{n} \frac{(\bar{X} - \mu)}{\sigma}}_{\text{Scaled Difference}} \rightarrow_d N(0,1)$$

Scaled Difference



“Asymptotically, the distribution of the scaled difference is approximately standard normal.”

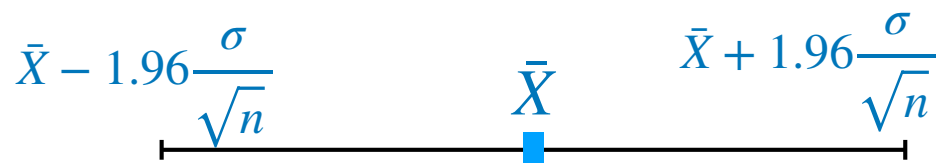
We know that that a standard normal RV is in $(-1.96, 1.96)$ with 95%.

So if I don't know what μ is but I observe $\bar{X}$ and the CLT states that the scaled difference is likely to be $(-1.96, 1.96)$, then...

$\bar{X} +$ upper bound of difference

$= \bar{X} + (\text{upper bound of scaled difference}) \times (\text{inverse of scaling term})$

$$= \bar{X} + 1.96 \frac{\sigma}{\sqrt{n}}$$



From CLT to Confidence intervals

The big idea: If we know that the difference between our estimator and estimand is (approximately) normally distributed, then we can construct an interval around our estimate such that the estimand will likely be in!

- The CLT states that $\sqrt{n} (\bar{X}_n - \mu) \rightarrow_d N(0, \sigma^2)$, which is equivalent to stating that

$$\lim_{n \rightarrow \infty} \Pr \left(\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - \mu \right) \leq c \right) = \Pr(\sigma Z \leq c) \text{ where } Z \sim N(0, 1).$$

$$= \Pr \left(\frac{1}{n} \sum_{i=1}^n X_i - \frac{1}{\sqrt{n}} c \leq \mu \right)$$

Lower bound

- Suppose we want a lower bound that holds with probability $1 - \alpha$. Then we must find c

$$\text{such that } \Pr(\sigma Z \leq c) = \Pr \left(Z \leq \frac{c}{\sigma} \right) = 1 - \alpha.$$

- Let $\tau_{1-\alpha}$ denote the $1 - \alpha$ quantile of the standard normal distribution, i.e.
 $\Pr(Z \leq \tau_{1-\alpha}) = 1 - \alpha.$

For $1 - \alpha = 0.025$,
 $\tau_{1-\alpha} = 1.96.$

- Then we should choose $c = \tau_{1-\alpha} \sigma.$

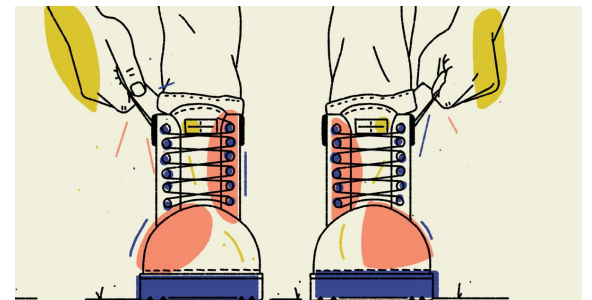
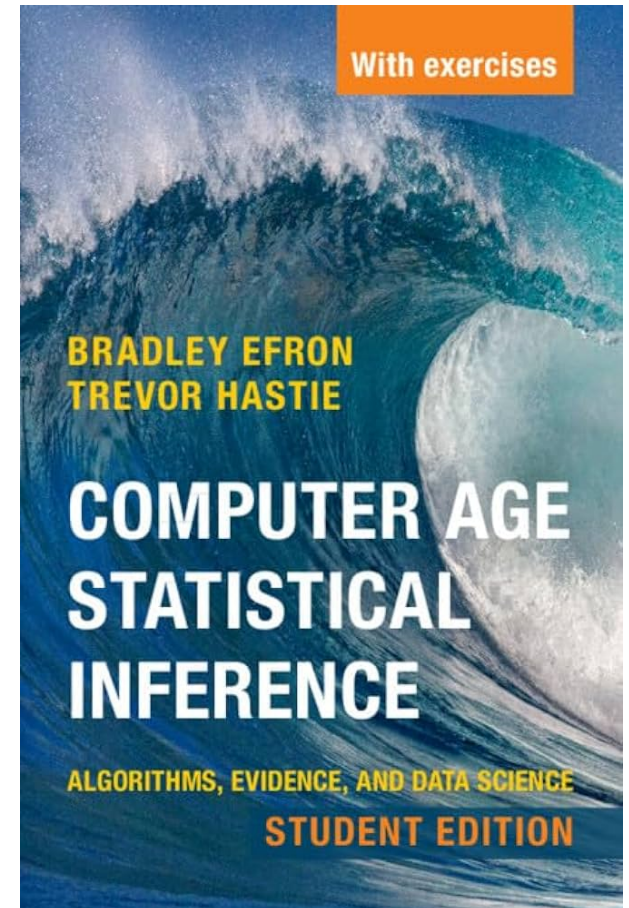
Quiz: How do we make smaller confidence intervals?

$$CI = \left[\frac{1}{n} \sum_{i=1}^n X_i - \frac{1}{\sqrt{n}} \tau_{1-\alpha} \sigma, \frac{1}{n} \sum_{i=1}^n X_i + \frac{1}{\sqrt{n}} \tau_{1-\alpha} \sigma \right]$$

- A: More observations
- B: Fewer observations
- C: Make the data less noisy
- D: Make the data more noisy

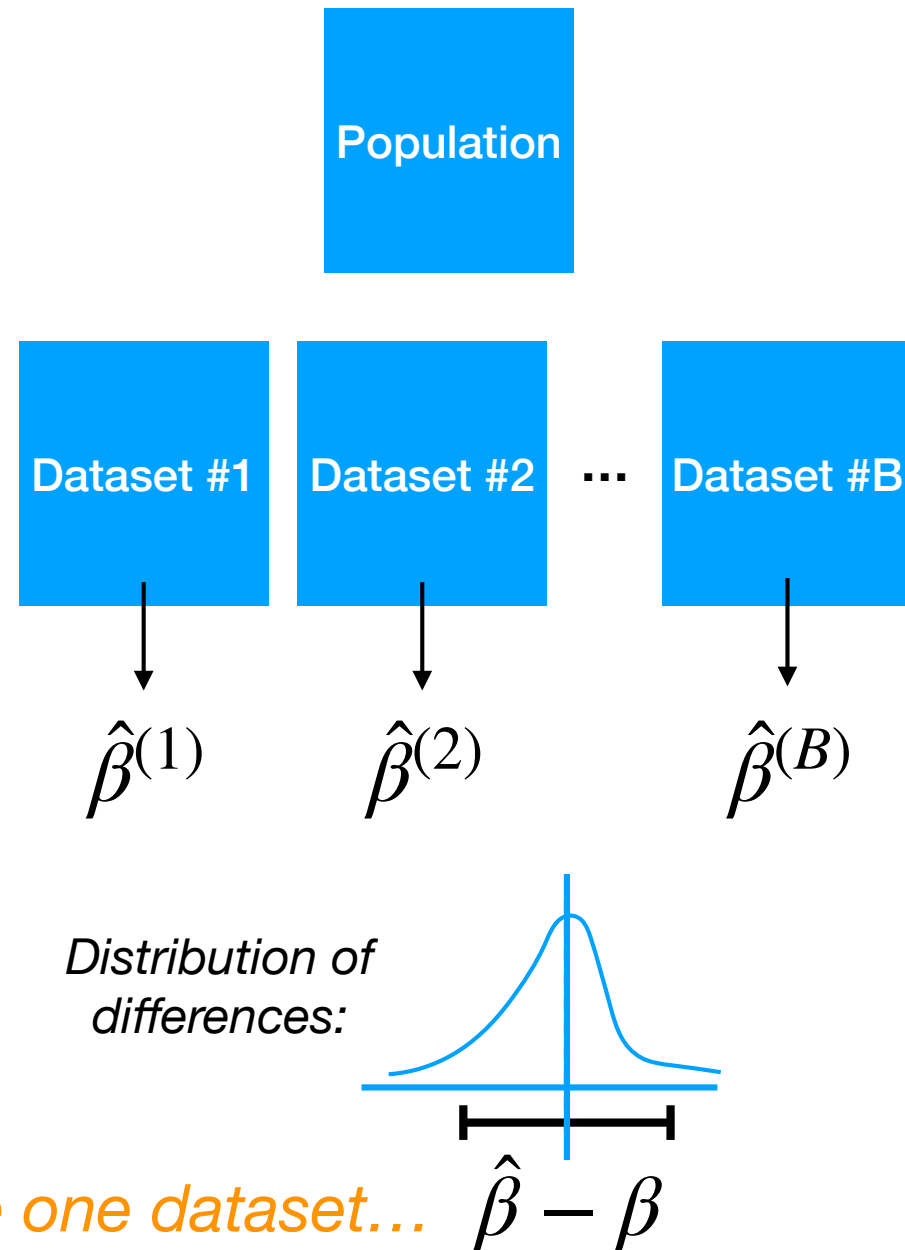
Bootstrap confidence intervals

- What if we don't have the math equation for the CI handy?
- **Resampling** methods provide a way to get a measure of uncertainty without having to derive the math!
- The **Bootstrap** is a powerful resampling method that constructs an approximation to the sampling distribution of an estimator without making strong assumptions about the data.
 - Completely automatic
 - No theoretical calculations
 - Available even for very complicated estimators!



Confidence intervals in the ideal world

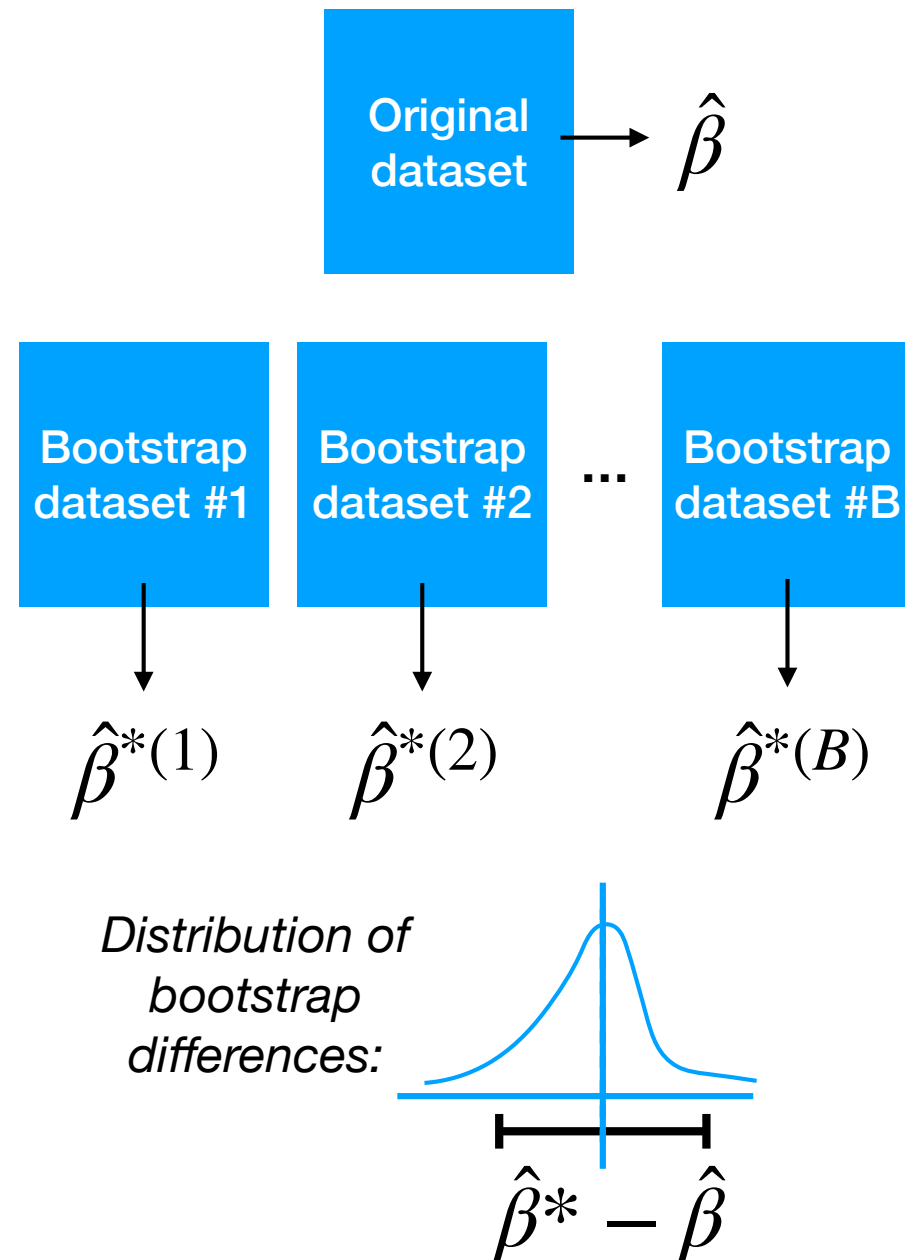
- In an **ideal world**, we would do the following to get confidence intervals:
 1. Sample multiple datasets from the population
 2. For each bootstrap sample, apply estimator.
 3. Construct the confidence interval using quantiles of estimates.



Usually, we only have one dataset... $\hat{\beta} - \beta$

Bootstrap confidence intervals

- We use the **bootstrap** to approximate the population (nonparametrically) using the original dataset:
- **Resample with replacement:** Create “bootstrap datasets” by resampling the original dataset *with replacement*.
 - Each bootstrap dataset should have the *same number of observations* as your original dataset.
- **Refit the model on the bootstrap datasets:** For each bootstrap sample, apply estimator. The distribution of $\hat{\beta}^* - \hat{\beta}$ approximates the sampling distribution of $\hat{\beta} - \beta$.



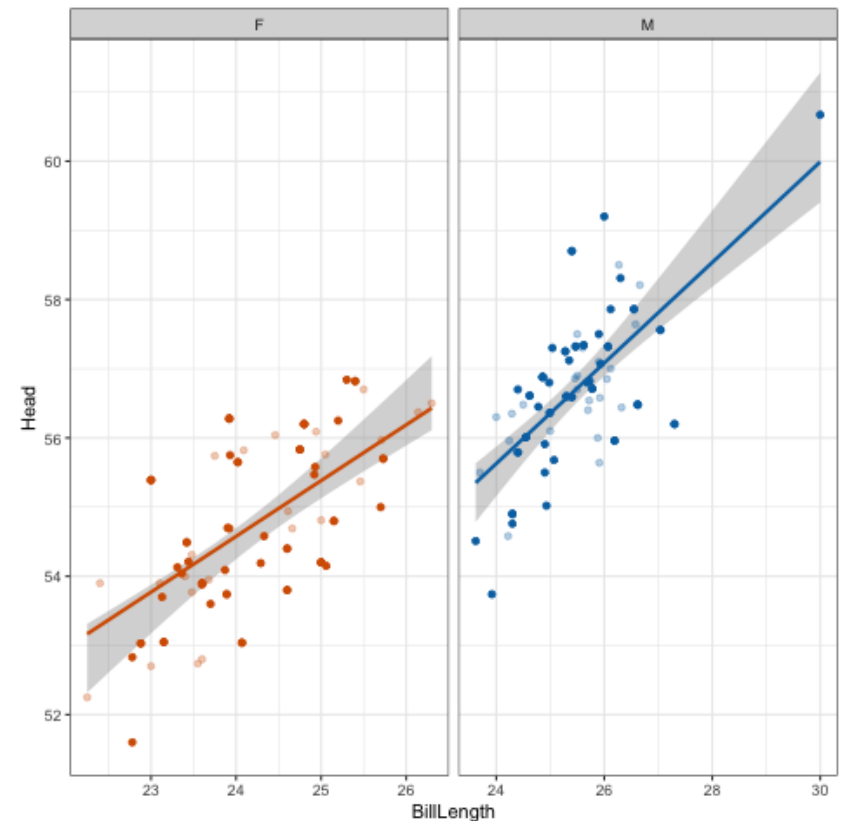
Bootstrap: How it works

$$\begin{aligned}1 - \alpha &= \Pr(q_{\alpha/2} \leq \tilde{T} \leq q_{1-\alpha/2}) \\&= \Pr(q_{\alpha/2} - \hat{T} \leq \tilde{T} - \hat{T} \leq q_{1-\alpha/2} - \hat{T}) \\&\approx \Pr(q_{\alpha/2} - \hat{T} \leq \hat{T} - t_0 \leq q_{1-\alpha/2} - \hat{T}) \\&= \Pr(q_{\alpha/2} - 2\hat{T} \leq -t_0 \leq q_{1-\alpha/2} - 2\hat{T}) \\&= \Pr(2\hat{T} - q_{1-\alpha/2} \leq t_0 \leq 2\hat{T} - q_{\alpha/2})\end{aligned}$$

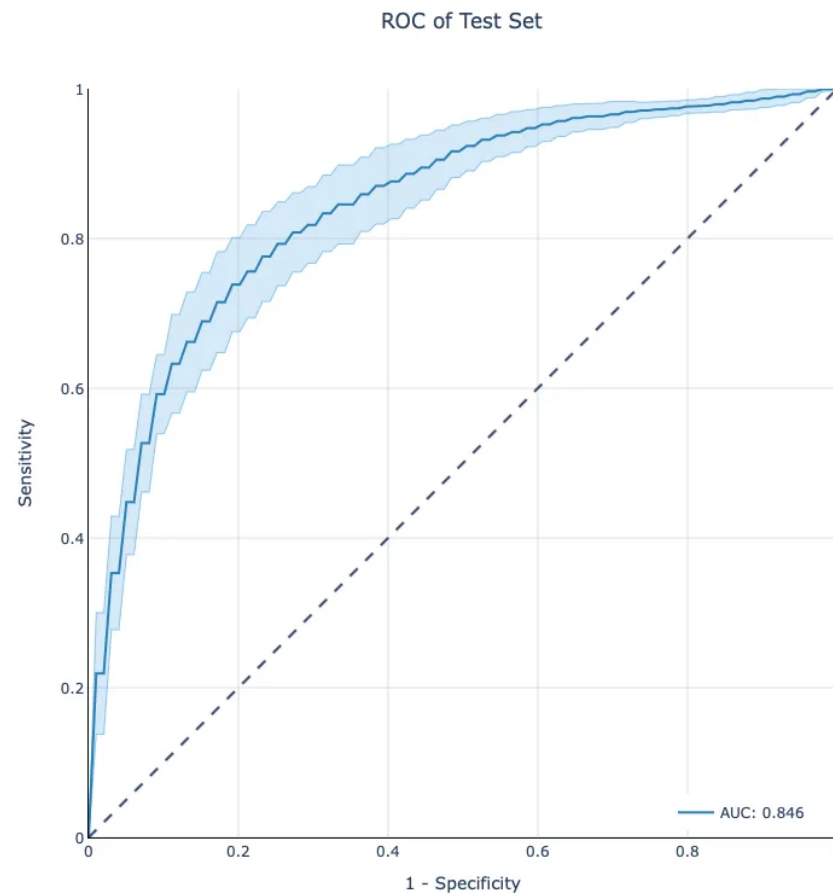
Basic Bootstrap CI $\Rightarrow C = [2\hat{T} - q_{\alpha/2}, 2\hat{T} - q_{1-\alpha/2}]$

Bootstrap confidence intervals for linear regression

- To get confidence intervals for linear regression for the dataset $\{(X_1, Y_1), \dots, (X_n, Y_n)\}$:
 - **Resample with replacement:** Create “bootstrap datasets” by resampling the original dataset with replacement.
 - $\{(X_1^{*(1)}, Y_1^{*(1)}), \dots, (X_n^{*(1)}, Y_n^{*(1)})\}$
 - ...
 - $\{(X_1^{*(B)}, Y_1^{*(B)}), \dots, (X_n^{*(B)}, Y_n^{*(B)})\}$
 - **Refit the model on the bootstrap datasets:** For each bootstrap sample, fit a linear regression model. The distribution of $\hat{\beta}_j^* - \hat{\beta}_j$ estimates approximates the distribution of $\hat{\beta}_j - \beta_j$.



How would you use the bootstrap for getting the CI for AUC?



Bootstrap computation

- The computation time for the Bootstrap grows linearly with the number of bootstrap datasets B .
 - In some sense, it's the price we pay when we don't derive the mathematical formulas.
- How many B to choose?
 - If B is too small, our approximation will be poor. There's no harm in choosing a big B , except for computation time.
 - Suggest choosing B large enough so that the confidence intervals are pretty stable. It's not uncommon to choose B to be around 2000.

The many versions of Bootstrap

- We just learned about the **nonparametric bootstrap** and constructed CI using **basic bootstrap intervals**.
- There are other types of bootstrap methods!
 - Parametric bootstrap
 - Wild bootstrap
 - Normal approximation
 - Percentile
 - Studentized (Bootstrap-t)
 - Bias Corrected and Accelerated (BCa)
- *Different bootstrap methods have different assumptions and properties. When using them, pay attention to what they are!*

The bootstrap is nifty but it doesn't always work!

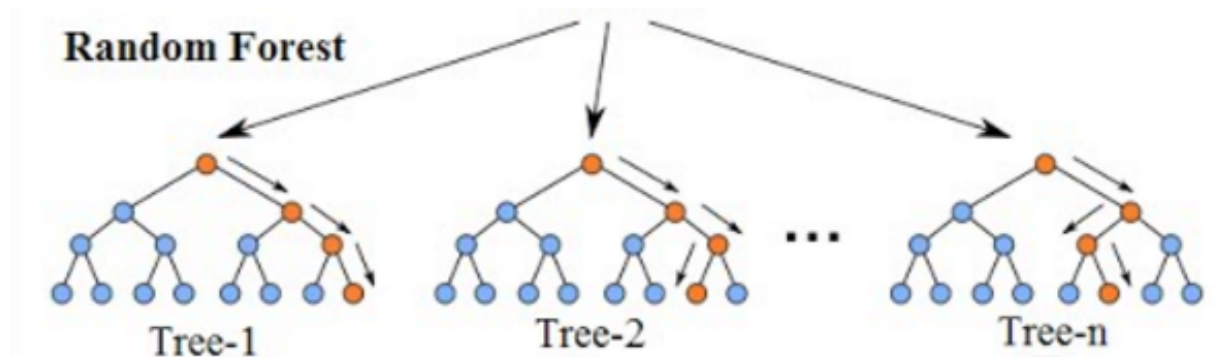
- Consider the distribution $Unif(0, \theta)$, where our goal is to find the maximum value of the uniform distribution.
- The bootstrap doesn't work here!
- *In general, the bootstrap only works for estimands that are sufficiently “smooth” with respect to small perturbations in the data generating process.*
- Suppose $\theta = 1$ and we sample $X_1, \dots, X_n \sim Unif[0, 1]$. The MLE is $M_n = \max\{X_1, \dots, X_n\}$. What happens when we use the bootstrap to construct a CI?

```
x <- runif(100)
is.covered <- function() {
  max.boot <- replicate(1e3, max(resample(x)))
  all(1 >= 2*max(x) - quantile(max.boot, 0.975),
      1 <= 2*max(x) - quantile(max.boot, 0.025))
}
sum(replicate(1000, is.covered()))
```

Coverage is 1.9%, not 95%!

Before we wrap up the bootstrap conversation...

Where else have you heard about bootstrap in ML?



Practical tips for confidence intervals

- **Understand the assumptions** under which our confidence intervals are valid. For example:
 - What assumptions are used to create CI for linear regression?
 - Before using the bootstrap, double check online to see if the bootstrap will give you a valid confidence interval.
- **Coding tips:** Scikit-learn often does not make it easy to get confidence intervals, but R does this automatically. You can also use statsmodels in python.
 - The bootstrap is a handy tool when a software package does not provide confidence intervals.

Lecture Outline

- Review of linear algebra
 - Vectorizing code
- Assumptions
- Solving Linear Regression
 - Closed-form solution
 - Gradient Descent
- Estimand vs estimate
- Uncertainty Quantification
 - Prediction interval vs confidence interval
 - Confidence intervals
- **Nonlinear Regression**

Quiz

- **Q:** What happens if we don't think the outcome Y follows a linear relationship with respect to X_1, X_2, X_3 ? Can we use linear regression to fit a nonlinear model?
- **A:** No.
- **B:** Yes, we can fit a nonlinear model by running linear regression respect to the new set of features $X_1, X_2, X_3, X_1 + X_2, X_2 + X_3$.
- **C:** Yes, we can fit a nonlinear model by running linear regression respect to the new set of features $X_1, X_2, X_3, X_1X_2, X_2X_3$.

Expanding the basis

- Change the **basis**, e.g. polynomial, spline, radial basis kernel

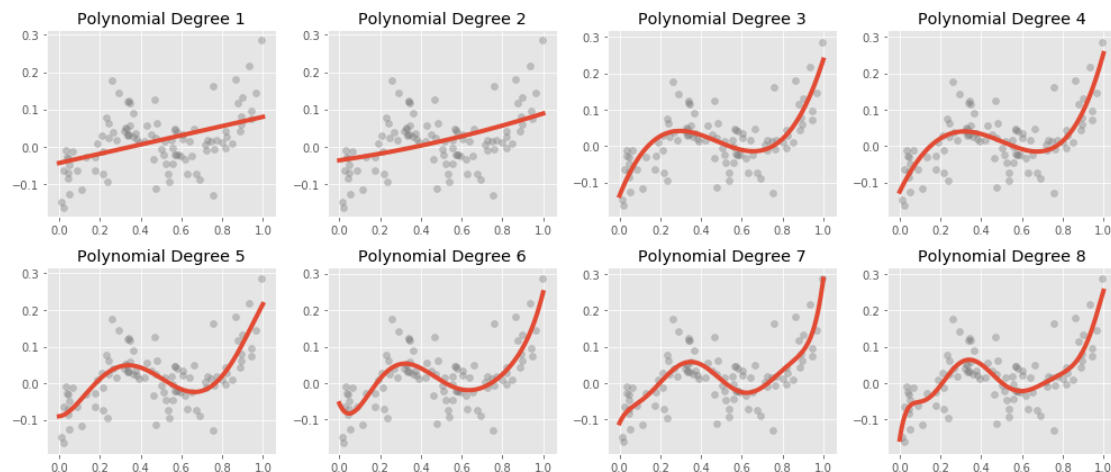
- Up to now, we've talked about the linear basis

$$\phi_j(x) = x_j.$$

$$h_{\theta}(x) = \sum_{j=0}^d \theta_j \underbrace{\phi_j(x)}_{\text{basis function}}$$

- 2nd order polynomial basis:
 $\{x_1, x_1^2, x_2, x_2^2, x_1x_2\}$

- **Sieve estimators** increase the size of the basis as data becomes more available. This lets us fit increasingly complex models, so it is a special type of nonparametric regression.



Useful Resources

- Linear algebra: MML chapter 2
- Vectorizing code:
 - <https://numpy.org/doc/stable/user/whatisnumpy.html#why-is-numpy-fast>
 - https://en.wikipedia.org/wiki/Array_programming
- Gradient descent:
 - Youtube: <https://youtube.com/watch?v=IHZwWFHWa-w>
- Bootstrap:
 - <https://www.nature.com/articles/nmeth.3414>
 - Efron B. and Tibshirani, R.J (1993), An Introduction of the Bootstrap.
 - Computer Age Statistical Inference, Chapters 10-11
 - Regression Methods in Biostatistics, Chapter 3.6